

VAULT & ERROR HANDLING

Securing Secrets and
Managing Task Failures Gracefully

Professional Reference Documentation

Version 1.0

Table of Contents

- 1. Introduction
- 2. Ansible Vault
 - 2.1 What Vault Protects
 - 2.2 Encrypting Files
 - 2.3 Encrypting Strings
 - 2.4 Viewing and Editing Vault Files
 - 2.5 Decrypting Files
 - 2.6 Vault IDs and Multiple Passwords
 - 2.7 Providing the Password
 - 2.8 Using Vault Variables in Playbooks
 - 2.9 Rekeying and Rotating Passwords
 - 2.10 Vault in CI/CD Pipelines
 - 2.11 What Vault Does Not Protect
- 3. Error Handling
 - 3.1 Default Failure Behaviour
 - 3.2 ignore_errors
 - 3.3 failed_when
 - 3.4 changed_when
 - 3.5 block / rescue / always
 - 3.6 any_errors_fatal
 - 3.7 max_fail_percentage
 - 3.8 Retries and until
- 4. Combining Vault and Error Handling
- 5. Best Practices
- 6. Quick Reference

1. Introduction

Two concerns run through every production playbook: protecting sensitive data and responding gracefully when something goes wrong. Ansible addresses both with first-class features.

- **Ansible Vault** — a tool and library that encrypts files and individual variables using AES-256. Encrypted secrets can be committed to the same Git repository as the playbooks that use them, eliminating the need for an external secrets manager for many deployments.
- **Error handling** — a set of task keywords (`ignore_errors`, `failed_when`, `changed_when`) and structural constructs (`block / rescue / always`) that let playbooks continue past non-critical failures, redefine what counts as failure or change, and roll back when something serious happens.

Together they let a single repository hold the entire automation environment — code, configuration, and secrets — while the playbooks themselves behave predictably under adverse conditions.

2. Ansible Vault

2.1 What Vault Protects

Ansible Vault encrypts the *contents* of a file or a single string using AES-256 in CTR mode with an HMAC-SHA256 signature for integrity. The result is a text file that can be safely stored alongside the rest of the automation repository.

Typical secrets managed with Vault:

- Database passwords and connection strings.
- API keys and tokens for cloud providers or third-party services.
- TLS private keys and certificates.
- SSH private keys used by service accounts.
- Application-specific credentials (SMTP, LDAP, OAuth).

Vault does not encrypt the entire repository. Only the specific files or values you designate are protected. Playbooks, roles, and templates remain plain text so they can be reviewed and diffed.

2.2 Encrypting Files

```
# Create a plain-text secrets file
cat > group_vars/all/vault.yml <<'EOF'
vault_db_password: "SuperSecret123!"
vault_api_key: "abcd1234-efgh-5678-ijkl-90mnopqrstuv"
vault_smtp_password: "mail-password"
EOF

# Encrypt it in place
ansible-vault encrypt group_vars/all/vault.yml

# Encrypt with a specific vault ID
ansible-vault encrypt --vault-id prod@prompt group_vars/all/vault.yml

# Encrypt multiple files at once
ansible-vault encrypt group_vars/all/vault.yml group_vars/production/vault.yml
```

After encryption, the file begins with a recognizable header and contains base64-encoded ciphertext. It is safe to commit:

```
$ANSIBLE_VAULT;1.1;AES256
66386439653236336... (many lines of base64) ...65313465346534
```

2.3 Encrypting Strings

`ansible-vault encrypt_string` encrypts a single value and prints it in YAML form, ready to paste into a variable file.

```
# Encrypt a string and print the YAML representation
ansible-vault encrypt_string 'SuperSecret123!' --name 'db_password'

# Output:
# db_password: !vault |
#   $ANSIBLE_VAULT;1.1;AES256
#   66386439653236336...
#   ...65313465346534

# Read the value from standard input (avoids shell history)
echo -n 'SuperSecret123!' | ansible-vault encrypt_string --stdin-name 'db_password'
```

The `!vault` tag tells Ansible to decrypt the value transparently when the variable is referenced.

2.4 Viewing and Editing Vault Files

```
# Read an encrypted file (decrypts to standard output)
ansible-vault view group_vars/all/vault.yml

# Edit an encrypted file in your editor
ansible-vault edit group_vars/all/vault.yml

# Create a new encrypted file directly
ansible-vault create group_vars/all/vault.yml
```

Warning: when editing an encrypted file, the unencrypted content is written to a temporary file in the system temporary directory. Ensure the temporary directory is on a trusted filesystem.

2.5 Decrypting Files

```
# Decrypt in place
ansible-vault decrypt group_vars/all/vault.yml

# Decrypt to a different file, leaving the encrypted original intact
ansible-vault decrypt --output /tmp/vault.yml group_vars/all/vault.yml

# Verify a file is encrypted without decrypting it
head -1 group_vars/all/vault.yml # should print $ANSIBLE_VAULT;...
```

2.6 Vault IDs and Multiple Passwords

A **vault ID** is a label attached to a password. It allows a single repository to hold secrets encrypted with different passwords — for example, development secrets encrypted with one password and production secrets with another.

```
# Encrypt with the "dev" vault ID
ansible-vault encrypt --vault-id dev@prompt group_vars/dev/vault.yml

# Encrypt with the "prod" vault ID
ansible-vault encrypt --vault-id prod@prompt group_vars/prod/vault.yml

# Decrypt a file encrypted with a specific ID
ansible-vault view --vault-id prod@prompt group_vars/prod/vault.yml
```

2.7 Providing the Password

Method	Example	Use case
Interactive prompt	<code>--ask-vault-pass</code>	Development / manual runs.
Password file	<code>--vault-password-file ~/.vault_pass</code>	Scripts and short-lived automation.
Executable script	<code>--vault-password-file ./vault_pass.sh</code>	Fetch from a secrets manager at runtime.
Vault ID + file	<code>--vault-id prod@~/.vault_prod</code>	Multiple environments.
Environment variable	<code>ANSIBLE_VAULT_PASSWORD_FILE=/path</code>	Global default for a session.

Table 1 — Methods for supplying vault passwords.

```
# ~/.vault_pass
MyVaultPassword

# Restrict permissions on the password file
chmod 600 ~/.vault_pass

# Add it to .gitignore (very important)
echo '.vault_pass' >> .gitignore
```

2.8 Using Vault Variables in Playbooks

```
# group_vars/all/vault.yml (encrypted)
vault_db_password: "SuperSecret123!"
vault_api_key: "abcd1234-efgh-5678-ijkl-90mnopqrstuv"

# group_vars/all/main.yml (plain text -- best practice)
db_password: "{{ vault_db_password }}"
api_key:     "{{ vault_api_key }}"

# site.yml
- name: Configure the application
  hosts: appservers
  become: true
  tasks:
    - name: Deploy application configuration
      ansible.builtin.template:
        src: app.conf.j2
        dest: /etc/myapp/app.conf
        owner: deploy
        group: deploy
        mode: '0640'
      notify: Restart myapp
```

2.9 Rekeying and Rotating Passwords

```
# Change the password on a single file
ansible-vault rekey group_vars/all/vault.yml

# Change from one vault ID to another
ansible-vault rekey --vault-id dev@oldpass --new-vault-id prod@newpass \
  group_vars/all/vault.yml
```

2.10 Vault in CI/CD Pipelines

```
# GitHub Actions example
- name: Run playbook
  env:
    ANSIBLE_VAULT_PASSWORD_FILE: /tmp/vault_pass
  run: |
    echo "$VAULT_PASS" > /tmp/vault_pass
    chmod 600 /tmp/vault_pass
    ansible-playbook -i inventory/production site.yml
```

2.11 What Vault Does Not Protect

- **The vault password itself.** If the password is weak or leaked, every encrypted file is compromised.
- **Runtime memory.** Decrypted values exist in memory during a playbook run and may appear in debug output or logs.
- **Files on disk after decryption.** Avoid `ansible-vault decrypt` in working trees.
- **Version history.** If a secret was ever committed in plain text, it remains in Git history even after encryption.
- **Access control.** Vault encrypts, it does not authorise.

3. Error Handling

3.1 Default Failure Behaviour

By default, Ansible stops executing tasks on a host as soon as one task fails. Other hosts in the play continue, but the failed host is removed from further task execution for the remainder of the play.

3.2 ignore_errors

```
- name: Check for the presence of an optional file
  ansible.builtin.command: cat /etc/myapp/optional.conf
  register: optional_file
  ignore_errors: true

- name: Show the result regardless of failure
  ansible.builtin.debug:
    msg: "File read returned {{ optional_file.rc }}"
```

Use ignore_errors sparingly. Ignoring a failure is usually a mistake. Prefer failed_when or a block/rescue construct.

3.3 failed_when

```
- name: Check whether a config file exists
  ansible.builtin.command: test -f /etc/myapp/app.conf
  register: config_check
  failed_when: config_check.rc not in [0, 1]

# rc 0 = file exists (success)
# rc 1 = file missing (not a failure -- we will create it)
# any other rc = genuine failure
```

3.4 changed_when

```
- name: Check the current status
  ansible.builtin.command: systemctl is-active myapp
  register: status
  changed_when: false    # inspection only -- nothing was modified

- name: Run the database migration
  ansible.builtin.command: /opt/myapp/bin/migrate
  register: migration
  changed_when: "'applied' in migration.stdout"
```

3.5 block / rescue / always

```
- name: Deploy with rollback on failure
hosts: appservers
become: true

tasks:
  - block:
    - name: Stop the application
      ansible.builtin.service:
        name: myapp
        state: stopped

    - name: Deploy the new version
      ansible.builtin.unarchive:
        src: myapp-{{ version }}.tar.gz
        dest: /opt/myapp

    - name: Start the application
      ansible.builtin.service:
        name: myapp
        state: started

  rescue:
    - name: Log the failure
      ansible.builtin.debug:
        msg: "Deployment failed -- initiating rollback"

    - name: Restart with the previous version
      ansible.builtin.service:
        name: myapp
        state: restarted

  always:
    - name: Send deployment notification
      ansible.builtin.debug:
        msg: "Deployment attempt completed"
```

Key behaviours:

- If any task in the block fails, execution jumps to rescue.
- The always section runs regardless of success or failure.
- If the rescue section itself fails, the play stops for that host after running always.

3.6 any_errors_fatal

```
- name: Rolling deployment -- abort on any failure
hosts: appservers
any_errors_fatal: true
serial: 1          # one host at a time
tasks:
  - name: Deploy the new version
    ansible.builtin.unarchive:
      src: myapp-{{ version }}.tar.gz
      dest: /opt/myapp
```

3.7 max_fail_percentage

```
- name: Deploy to the fleet, tolerating up to 25% failures per batch
  hosts: appservers
  serial: 20
  max_fail_percentage: 25
  tasks:
    - name: Deploy the new version
      ansible.builtin.unarchive:
        src: myapp-{{ version }}.tar.gz
        dest: /opt/myapp
```

3.8 Retries and until

```
- name: Wait for the API to respond
  ansible.builtin.uri:
    url: http://localhost:8080/health
    status_code: 200
  register: result
  until: result.status == 200
  retries: 10
  delay: 5
  # Total wait budget: 10 * 5 = 50 seconds
```

4. Combining Vault and Error Handling

```
---
- name: Deploy the application with rollback
  hosts: appservers
  become: true
  serial: 1
  any_errors_fatal: true

  vars_files:
    - group_vars/all/vars.yml          # plain -- references vault vars
    - group_vars/all/vault.yml         # encrypted with ansible-vault

  tasks:
    - name: Verify the vault is readable
      ansible.builtin.assert:
        that:
          - vault_db_password is defined
          - vault_api_key is defined
        fail_msg: "Vault variables are missing -- is the vault password correct?"

    - block:
      - name: Stop the application
        ansible.builtin.service:
          name: myapp
          state: stopped

      - name: Deploy the new release
        ansible.builtin.unarchive:
          src: "myapp-{{ version }}.tar.gz"
          dest: /opt/myapp/releases

      - name: Start the application
        ansible.builtin.service:
          name: myapp
          state: started
```

```
- name: Verify the application is healthy
  ansible.builtin.uri:
    url: "http://localhost:{{ app_port }}/health"
    status_code: 200
  register: health
  until: health.status == 200
  retries: 10
  delay: 3

rescue:
  - name: Notify the team of the failure
    ansible.builtin.debug:
      msg: "Deployment of {{ version }} failed on {{ inventory_hostname }}"

  - name: Restart the application with the previous release
    ansible.builtin.service:
      name: myapp
      state: restarted

  - name: Fail the play after rollback
    ansible.builtin.fail:
      msg: "Deployment failed and was rolled back"

always:
  - name: Record the deployment attempt
    ansible.builtin.lineinfile:
      path: /var/log/deployments.log
      line: "{{ ansible_date_time.iso8601 }} {{ version }} {{ inventory_hostname }}"
      create: true
```

5. Best Practices

Ansible Vault

- **Never commit the vault password.** Add it to `.gitignore` and store it in a dedicated secrets manager.
- **Use vault IDs.** Separate dev, staging, and production passwords.
- **Follow the vault_ naming convention.** Encrypt variables named `vault_*`.
- **Encrypt files, not individual strings, for related secrets.**
- **Rotate vault passwords.** Use `ansible-vault rekey` periodically.
- **Never decrypt files in the repository.** Use `view` or `edit`.

Error Handling

- **Avoid `ignore_errors`.** It masks real problems.
- **Always set `changed_when` on command and shell tasks.**
- **Prefer `failed_when` over ignoring errors.** Define the precise condition that constitutes failure.
- **Use `block/rescue/always` for multi-step operations.**
- **Add retries to network-dependent tasks.**
- **Use `any_errors_fatal` for fleet-wide deployments.**

- **Log every failure.** Record deployments and rollbacks.

6. Quick Reference

Ansible Vault

Command	Purpose
<code>ansible-vault encrypt file</code>	Encrypt a plain file in place.
<code>ansible-vault decrypt file</code>	Decrypt a file in place (avoid in repositories).
<code>ansible-vault view file</code>	Read an encrypted file without modifying it.
<code>ansible-vault edit file</code>	Edit an encrypted file in \$EDITOR.
<code>ansible-vault create file</code>	Create a new encrypted file.
<code>ansible-vault rekey file</code>	Change the password on an encrypted file.
<code>ansible-vault encrypt_string</code>	Encrypt a single value and print it as YAML.
<code>--ask-vault-pass</code>	Prompt for the vault password.
<code>--vault-password-file FILE</code>	Read the password from FILE (or from an executable).
<code>--vault-id ID@SOURCE</code>	Use a named vault ID with a source (prompt, file, script).

Table 2 — Ansible Vault quick reference.

Error Handling

Keyword	Effect
<code>ignore_errors: true</code>	Mark a task as failed but continue the play.
<code>failed_when: EXPR</code>	Redefine the failure condition.
<code>failed_when: false</code>	Never fail (equivalent to <code>ignore_errors</code>).
<code>changed_when: EXPR</code>	Redefine when a task reports a change.
<code>changed_when: false</code>	Never report a change (inspection tasks).
<code>block / rescue / always</code>	Structured exception handling.
<code>any_errors_fatal: true</code>	Abort the play for every host if any host fails.
<code>max_fail_percentage: N</code>	Tolerate up to N% failures per batch.
<code>until / retries / delay</code>	Retry a task until a condition succeeds.
<code>register: VAR</code>	Save the result for later inspection.
<code>fail: msg=...</code>	Explicitly fail the play.
<code>--force-handlers</code>	Run handlers even after a task fails.

Table 3 — Error handling quick reference.

This document describes Ansible Core behaviour as of version 2.14 and later. Vault format versions and error-handling keywords may evolve between releases; always consult the official documentation for the version in use.